

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a\agent-a033188df9b8314fc.jsonl`
- SHA-256: `82d8e5ceda1d43fd260409c2786fedffa498444d6bc8092faed4595958e284f2`
- Source modified: `2026-07-06T07:57:15+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:55:14.238Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).

It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.

Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:

- db.py: `_migration_004_destinations_and_routing` (backfills destinations from `filter_config_deprecated`, `routing_rules` per source, `delivery_attempts` from `videos_deprecated.forwarded=1`); `repos` `upsert_destination`, `get_destination`, `upsert_delivery_attempt`, `record_delivery` (UPSERT with `attempt_count` increment), `get_delivery_attempt`.
- delivery.py: `forward_to_target/send_link_message/deliver` gain an optional `destination_id` kwarg; when set, `record_delivery` writes a `DeliveryAttempt`; when None (all current live callers) falls back to the `mark_forwarded` no-op.
- models.py: `Destination`, `RoutingRule`, `DeliveryStatus`, `DeliveryAttempt`.

Note: routing is NOT yet wired into the live path (`telethon_client/manager` still call `deliver()` with `destination_id=None`); `destination_id` is currently exercised only by tests. So `record_delivery` issues are LATENT until Track D, but ship now.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: `C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>`. Build schema via `db._apply_schema(conn)` on `aiosqlite ":memory:"`; or `db.init_db(path)` on a temp file seeded with a legacy v1 schema (`users/channels/videos/filter_config`) to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns `duration_s,width,height,size_bytes,mime_type` or migration 002's column-copy fails (that's a test-harness requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): `ruff check clean`; `pytest 451` passed @ 95.75% (floor 80%); `ruff format --check flags commands.py` ONLY because this branch predates the merged #36 format fix (`commands.py` is NOT in this PR) ? do NOT report that as a #37 finding.

- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from LATENT (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding. Rank severe first.

- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug, narrow trigger), medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic triggers.

ADVERSARIALLY VERIFY this single finding ? try to REFUTE it. Read the exact cited code, trace real control flow, and run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour (note live vs latent). REFUTED if the code handles it, the trigger can't occur, or it's pre-existing/not in this diff. PLAUSIBLE if you can't fully settle it. Give corrected severity.

FINDING (dim tests):

title: Migration 004: account with forwarded videos but no filter_config row is untested and silently drops delivery history
severity(claimed): medium | live
where: src/tg_video_filter/db.py:685
summary: Migration 004 backfills destinations only from filter_config_deprecated (db.py:690-704). An account with forwarded videos but no filter_config row gets no destination, no routing rule, and therefore NO delivery_attempts backfill - its forwarded history is dropped on upgrade. No test covers this case; migration tests only cover accounts that have a filter_config.
failure_scenario: On a production upgrade, any account that has watched channels and forwarded videos but never wrote a filter_config row (e.g. defaults-only user) loses all its forwarded->delivered history: no destination is created, so the JOINS in step 3 (db.py:733-747) produce zero rows for that account. My repro (user 333: channel + forwarded message 99, no filter_config) confirmed 333 got zero destinations, zero routing_rules, and its forwarded video produced no delivery_attempt, while users 111/222 (with filter_config) backfilled correctly.
evidence: Repro against the migration ladder (init_db on a legacy DB) printed destinations only for account_id 111 and 222; account 333 absent, and delivery_attempts contained only content_item_id=1 (111's video), not 333's forwarded message 99. coverage term-missing flags db.py:686 (the `not filter\_config\_deprecated` early return) uncovered.
test_migration_004_backfills_destinations_and_attempts (test_db.py:1591) only seeds accounts that have filter_config.
recommendation: Decide the intended behavior for filter_config-less accounts (likely: default them to a 'me'/saved destination so forwarded history survives) and add a migration test that seeds an account with a forwarded video and NO filter_config, asserting either a defaulted destination+attempt or an explicit documented skip. As written this is an undocumented, untested data-drop on upgrade.

2. assistant (2026-07-06T07:55:16.690Z)

I'll verify this finding by reading the actual migration code and running a repro.

3. user (2026-07-06T07:55:17.416Z)

```
660         # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661         #     rollback safety net (ADR 0010 pattern).
662         await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663         await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
664         await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
665
666
667     @migration(4)
668     async def _migration_004_destinations_and_routing(conn: aiomysql.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671         Creates a default destination row for each account that has a
672         ``filter_config_deprecated.delivery_target`` value, then creates a default
```

```

673 routing rule (source ? default policy ? destination) for each source.
674 Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675 table (one ``delivered`` attempt per previously-forwarded video to the
676 account's default destination).
677
678 Idempotent: skips accounts whose default destination already exists.
679 """
680 # Gate: if destinations already have data, this migration has run.
681 async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682     if (await cur.fetchone())[0] > 0:
683         return
684
685 if not await _table_exists(conn, "filter_config_deprecated"):
686     return
687
688 # 1. Backfill default destinations from filter_config_deprecated.
689 # Parse the JSON delivery_target, defaulting to "me".
690 await conn.execute(
691     """
692     INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
693     SELECT user_id, 'telegram',
694            COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
695            CASE
696                WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
697                THEN 'saved'
698                ELSE 'channel'
699            END,
700            0,
701            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
702     FROM filter_config_deprecated
703     """
704 )
705
706 # 2. Create a default routing rule per source (source ? default policy ? default
destination).
707 await conn.execute(
708     """
709     INSERT OR IGNORE INTO routing_rules
710     (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
711     SELECT s.account_id, s.id, p.id, d.id, 1, 0,
712            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
713     FROM sources s
714     JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
715     JOIN destinations d ON d.account_id = s.account_id
716     WHERE d.target = (
717         SELECT COALESCE(
718             json_extract(fcd.config_json, '$.delivery_target'), 'me'
719         )
720     FROM filter_config_deprecated fcd
721     WHERE fcd.user_id = s.account_id
722     LIMIT 1
723     )
724     """
725 )
726
727 # 3. Backfill delivery_attempts from previously-forwarded videos.
728 await conn.execute(
729     """
730     INSERT OR IGNORE INTO delivery_attempts
731     (content_item_id, destination_id, route_id, status, attempt_count,
732     created_at, delivered_at)
733     SELECT ci.id, d.id, rr.id, 'delivered', 1,

```

```

734         vd.seen_at, vd.seen_at
735     FROM videos_deprecated vd
736     JOIN content_items ci
737         ON ci.account_id = vd.user_id
738         AND ci.provider = 'telegram'
739         AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
740     JOIN sources sc
741         ON sc.id = ci.source_id
742         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
743     JOIN destinations d ON d.account_id = vd.user_id
744     JOIN routing_rules rr
745         ON rr.account_id = vd.user_id
746         AND rr.source_id = sc.id
747         AND rr.destination_id = d.id
748     WHERE vd.forwarded = 1
749     """
750 )
751
752
753 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
754     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
755     async with conn.execute(
756         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
757     ) as cur:
758         return await cur.fetchone() is not None
759
760
761 # --- helpers -----
762
763
764 def _now() -> datetime:
765     """Timezone-aware UTC 'now', used for policy timestamps."""
766     return datetime.now(UTC)
767
768
769 def _safe_int_cast(value: object, field_name: str) -> int:
770     """Cast a provider_source_id / provider_message_id / document_id to int.
771
772     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
773     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
774     shapes (which carry Telegram-specific int ids); this function raises a clear
775     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
776     (ADR 0011 provider-neutrality guard ? review finding #5).
777     """
778     try:
779         return int(value) # type: ignore[arg-type]

```

4. assistant (2026-07-06T07:55:19.831Z)

Now let me look at the test to confirm what's covered, and check the schema for the tables involved.

5. user (2026-07-06T07:55:20.514Z)

```

1580     )
1581     await db.record_delivery(conn, content_item_id=item.id, destination_id=dest.id,
1582                             success=True)
1583     attempt = await db.get_delivery_attempt(conn, item.id, dest.id)
1584     assert attempt is not None
1585     assert attempt.status == DeliveryStatus.DELIVERED
1586     assert attempt.attempt_count == 2 # failed + succeeded
1587
1588 # --- Track C: Migration 004 -----
1589

```

```

1590
1591     async def test_migration_004_backfills_destinations_and_attempts(tmp_path: object) ->
None:
1592         """Migration 004 backfills default destinations from filter_config and
1593         creates delivery_attempts for previously-forwarded videos."""
1594         db_path = str(tmp_path) + "/mig004.db" # type: ignore[operator]
1595         # Build a pre-ADR-0011 legacy DB with a user, channel, video, and filter_config.
1596         legacy_schema = """
1597         CREATE TABLE users (id INTEGER PRIMARY KEY, phone TEXT, enabled INTEGER NOT NULL
DEFAULT 1, created_at TEXT NOT NULL);
1598         CREATE TABLE channels (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT
NULL REFERENCES users(id), channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
UNIQUE (user_id, channel_id));
1599         CREATE TABLE videos (id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL
REFERENCES users(id), channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL, duration_s
INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER, mime_type TEXT, matched INTEGER NOT
NULL DEFAULT 0, forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL, UNIQUE (user_id,
channel_id, message_id));
1600         CREATE TABLE filter_config (user_id INTEGER PRIMARY KEY REFERENCES users(id),
config_json TEXT NOT NULL);
1601         """
1602         async with aiosqlite.connect(db_path) as conn:
1603             await conn.executescript(legacy_schema)
1604             await conn.execute("INSERT INTO users VALUES
(111, '+1', 1, '2024-01-01T00:00:00Z')")
1605             await conn.execute(
1606                 "INSERT INTO channels (user_id, channel_id, title, seen_first_at) VALUES
(111, -1001, 'Ch', '2024-01-01T00:00:00Z')")
1607             )
1608             await conn.execute(
1609                 "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
height, size_bytes, mime_type, matched, forwarded, seen_at) "
1610                 "VALUES (111, -1001, 42, 60, 1920, 1080, 1000, 'video/mp4', 1, 1,
'2024-01-01T00:00:00Z')")
1611             )
1612             # filter_config with @my-feed delivery_target
1613             await conn.execute(
1614                 'INSERT INTO filter_config (user_id, config_json) VALUES (111,
\'{"delivery_target": "@my-feed", "delivery_mode": "forward"}\')')
1615             )
1616             await conn.commit()
1617
1618         await db.init_db(db_path)
1619
1620         async with aiosqlite.connect(db_path) as conn:
1621             # Destination backfilled from filter_config.
1622             async with conn.execute(
1623                 "SELECT target, kind FROM destinations WHERE account_id=111"
1624             ) as cur:
1625                 d = await cur.fetchone()
1626                 assert d == ("@my-feed", "channel")
1627
1628             # Routing rule created (source ? default policy ? destination).
1629             async with conn.execute(
1630                 "SELECT COUNT(*) FROM routing_rules WHERE account_id=111 AND enabled=1"
1631             ) as cur:
1632                 assert (await cur.fetchone())[0] >= 1
1633
1634             # Delivery attempt backfilled for the forwarded video.
1635             async with conn.execute("SELECT status, attempt_count FROM delivery_attempts")
as cur:
1636                 da = await cur.fetchone()
1637                 assert da is not None
1638                 assert da == ("delivered", 1)
1639

```

```

1640         # user_version bumped to latest migration.
1641         async with conn.execute("PRAGMA user_version") as cur:
1642             assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1643
1644
1645     async def test_migration_004_is_idempotent(tmp_path: object) -> None:
1646         """Migration 004 is a no-op on second run (destinations already populated)."""
1647         db_path = str(tmp_path) + "/idem004.db" # type: ignore[operator]
1648         await db.init_db(db_path)
1649         await db.init_db(db_path) # second run
1650         async with aiosqlite.connect(db_path) as conn:
1651             async with conn.execute("PRAGMA user_version") as cur:
1652                 assert (await cur.fetchone())[0] == len(db._MIGRATIONS)
1653             assert await db._table_exists(conn, "destinations")
1654             assert await db._table_exists(conn, "routing_rules")
1655             assert await db._table_exists(conn, "delivery_attempts")
1656

```

6. assistant (2026-07-06T07:55:25.098Z)

The test confirms only accounts with a filter_config row are seeded. Now the key question for the finding: can an account have forwarded videos but NO filter_config row? Let me check how filter_config was populated in the legacy schema and migrations 001-003.

7. user (2026-07-06T07:55:26.200Z)

```

560         (account_id, source_id, provider, provider_message_id, content_type,
561          document_id, duration_s, width, height, size_bytes, mime_type,
562          dedup_key, matched, seen_at)
563     SELECT
564         v.user_id,
565         s.id,
566         'telegram',
567         CAST(v.message_id AS TEXT),
568         'video',
569         CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
570         v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
571         'tg:msg:' || s.id || ':' || v.message_id, -- Tier-0 (always unique)
572         v.matched,
573         v.seen_at
574     FROM videos v
575     JOIN sources s
576         ON s.account_id = v.user_id
577         AND s.provider = 'telegram'
578         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
579     """"
580 )
581
582 # Phase B: promote one row per (account, document_id) to tg:doc:.
583 await conn.execute(
584     """"
585     UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
586     WHERE document_id IS NOT NULL
587         AND dedup_key LIKE 'tg:msg:%'
588         AND id IN (
589             SELECT MIN(id) FROM content_items
590             WHERE document_id IS NOT NULL AND dedup_key LIKE 'tg:msg:%'
591             GROUP BY account_id, document_id
592         )
593     """"
594 )
595
596 # Phase C: promote one non-doc row per (account, fingerprint) to tg:fp:.
597 await conn.execute(
598     """"

```

```

599         UPDATE content_items SET dedup_key =
600             'tg:fp:' || size_bytes || ':' || duration_s || ':'
601             || height || ':' || width
602     WHERE document_id IS NULL
603         AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
604         AND height IS NOT NULL AND width IS NOT NULL
605         AND dedup_key LIKE 'tg:msg:%'
606         AND id IN (
607             SELECT MIN(id) FROM content_items
608             WHERE document_id IS NULL
609                 AND size_bytes IS NOT NULL AND duration_s IS NOT NULL
610                 AND height IS NOT NULL AND width IS NOT NULL
611                 AND dedup_key LIKE 'tg:msg:%'
612             GROUP BY account_id, size_bytes, duration_s, height, width
613         )
614     """
615 )
616
617 # 3. Backfill policies from filter_config: one 'default' policy per account
618 #     (Option 2A). is_default=1 so the legacy load_filter_config shim reads
619 #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620 await conn.execute(
621     """
622     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
623                                     created_at, updated_at)
624     SELECT user_id, 'default', config_json, 1,
625            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
626            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
627     FROM filter_config
628     """
629 )
630
631 # 4. Create the policies default-partial index (sig after backfill so the
632 #     UNIQUE constraint only applies once rows are populated). On fresh DBs
633 #     _SCHEMA creates it; on migrated DBs this is the first time.
634 await conn.execute(
635     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
636     "ON policies(account_id) WHERE is_default = 1"
637 )
638
639 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
640 #     produce a content_items row. A mismatch signals fingerprint collisions
641 #     where two distinct videos shared (size,dur,h,w) ? the losers stay at
642 #     tg:msg: keys and are still present. Log a warning so the operator can
643 #     investigate; the data is preserved in *_deprecated tables.
644 async with conn.execute("SELECT COUNT(*) FROM videos") as cur:
645     vc = (await cur.fetchone())[0]
646 async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
647     cic = (await cur.fetchone())[0]
648 if vc != cic:
649     import logging
650
651     _logger = logging.getLogger(__name__)
652     _logger.warning(
653         "migration_003: %d videos rows produced %d content_items rows "
654         "(%d gap). Colliding items remain in videos_deprecated.",
655         vc,
656         cic,
657         vc - cic,
658     )
659
660 # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661 #     rollback safety net (ADR 0010 pattern).
662 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")

```

```
664         await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
```

8. assistant (2026-07-06T07:55:31.104Z)

Now I understand the migration chain. The key point: `sources` is backfilled from `channels`, and `content\_items` from `videos`, independent of `filter\_config`. So an account with channels+videos but no filter_config WILL have sources and content_items, but NO destination (step 1 only reads filter_config_deprecated). This confirms the mechanism. Let me verify whether such an account can realistically exist by checking how filter_config is written in the live code.

9. user (2026-07-06T07:55:33.077Z)

```
src\tg_video_filter\bootstrap.py:75:         await db.save_filter_config(conn, user.id,
DEFAULT_FILTER_CONFIG)
src\tg_video_filter\commands.py:198:     (Saved Messages). It persists config changes via
`db.save_filter_config`
src\tg_video_filter\commands.py:262:         current_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:296:         current_cfg = await db.load_filter_config(conn,
user.id)
src\tg_video_filter\commands.py:374:         cfg = await db.load_filter_config(conn, user.id)
src\tg_video_filter\commands.py:401:     cfg = await db.load_filter_config(conn, user_id)
src\tg_video_filter\commands.py:416:     await db.save_filter_config(conn, user_id, cfg)
src\tg_video_filter\db.py:81:CREATE TABLE IF NOT EXISTS filter_config (
src\tg_video_filter\db.py:207:     # ``channels`` / ``videos`` / ``filter_config`` tables
(they're needed for
src\tg_video_filter\db.py:215:         await conn.execute("DROP TABLE IF EXISTS filter_config")
src\tg_video_filter\db.py:357:     data in ``channels`` / ``videos`` / ``filter_config`` is
copied verbatim,
src\tg_video_filter\db.py:363:     FK contract: ``channels`` / ``videos`` / ``filter_config`` are
rebuilt with
src\tg_video_filter\db.py:425:     # Rebuild channels / videos / filter_config so their FK
constraints point
src\tg_video_filter\db.py:472:         "filter_config",
src\tg_video_filter\db.py:474:         CREATE TABLE filter_config (
src\tg_video_filter\db.py:499:     # filter_config tables were already rebuilt above with FKs to
accounts(id),
src\tg_video_filter\db.py:512:     ``channels`` / ``videos`` / ``filter_config`` tables and
renames the
src\tg_video_filter\db.py:617:     # 3. Backfill policies from filter_config: one 'default'
policy per account
src\tg_video_filter\db.py:618:     #     (Option 2A). is_default=1 so the legacy
load_filter_config shim reads
src\tg_video_filter\db.py:627:         FROM filter_config
src\tg_video_filter\db.py:664:     await conn.execute("ALTER TABLE filter_config RENAME TO
filter_config_deprecated")
src\tg_video_filter\db.py:672:     ``filter_config_deprecated.delivery_target`` value, then
creates a default
src\tg_video_filter\db.py:685:     if not await _table_exists(conn, "filter_config_deprecated"):
src\tg_video_filter\db.py:688:     # 1. Backfill default destinations from
filter_config_deprecated.
src\tg_video_filter\db.py:702:         FROM filter_config_deprecated
src\tg_video_filter\db.py:720:         FROM filter_config_deprecated fcd
src\tg_video_filter\db.py:1662:async def load_filter_config(conn: aiosqlite.Connection, user_id:
int) -> FilterConfig:
src\tg_video_filter\db.py:1665:     ADR 0011 (Option 2A): ``filter_config`` is renamed to
src\tg_video_filter\db.py:1666:     ``filter_config_deprecated``; this reads ``policies WHERE
is_default=1``
src\tg_video_filter\db.py:1675:async def save_filter_config(conn: aiosqlite.Connection, user_id:
int, cfg: FilterConfig) -> None:
src\tg_video_filter\db.py:1771:     "load_filter_config",
src\tg_video_filter\db.py:1772:     "save_filter_config",
src\tg_video_filter\manager.py:140:         cfg = await db.load_filter_config(conn,
session.account_id)
```

```
src\tg_video_filter\models.py:120:# Option 2A: ``policies`` generalizes ``filter_config``.
src\tg_video_filter\models.py:189:     Generalizes the legacy per-account ``filter_config`` row
so one account can
src\tg_video_filter\models.py:193:     ``load_filter_config`` / ``save_filter_config`` shims
read/write that row.
src\tg_video_filter\models.py:214:# destinatinons (backfilled from
filter_config.delivery_target), writes
src\tg_video_filter\models.py:302:     Stored in the DB as a JSON blob
(``filter_config.config_json``) so adding a
src\tg_video_filter\filters\__init__.py:10:     engine = FilterEngine(filter_config)
```

10. assistant (2026-07-06T07:55:35.326Z)

Let me check bootstrap.py to see whether every account gets a filter_config written at bootstrap time.

11. user (2026-07-06T07:55:36.603Z)

```
55
56     Pure DB operation ? unit-tested. Returns the persisted user (legacy view).
57
58     ADR 0010: onboarding now writes one row to ``accounts`` (the product
59     identity) and one to ``telegram_sessions`` (the MTProto credential), rather
60     than a single conflated ``users`` row. The legacy ``User`` shape is
61     preserved as the return type so existing callers/tests are unaffected.
62     """
63     async with aiosqlite.connect(db_path) as conn:
64         await db._apply_schema(conn)
65         await db.upsert_account(conn, Account(id=user.id, telegram_id=user.id))
66         await db.upsert_telegram_session(
67             conn,
68             TelegramSession(
69                 account_id=user.id,
70                 phone=user.phone,
71                 session_path=f"user_{user.id}.session",
72                 enabled=user.enabled,
73             ),
74         )
75         await db.save_filter_config(conn, user.id, DEFAULT_FILTER_CONFIG)
76         persisted = await db.get_user(conn, user.id)
77         assert persisted is not None # just inserted
78         return persisted
79
80
81     async def bootstrap( # pragma: no cover - interactive Telethon login (real connection +
human)
82         settings: Settings,
83         *,
84         phone_prompt: PhonePrompt = _default_phone_prompt,
85         code_prompt: CodePrompt = _default_code_prompt,
86         password_prompt: PasswordPrompt = _default_password_prompt,
87     ) -> User:
88         """Run the interactive onboarding for one user.
89
```

12. assistant (2026-07-06T07:55:43.999Z)

This is the key. At bootstrap, every account gets `save\_filter\_config(conn, user.id, DEFAULT\_FILTER\_CONFIG)` written. So in the live path, every onboarded account has a filter_config row. But ? `save\_filter\_config` post-migration writes to `policies`, not `filter\_config`. The question for a legacy (pre-ADR-0011) DB being upgraded is: did the OLD bootstrap always write a filter_config row?

The legacy `filter\_config` table has `user\_id PRIMARY KEY`. Bootstrap at line 75 writes

DEFAULT_FILTER_CONFIG. So on any DB that went through this bootstrap, every account has a filter_config row. Let me verify there is no other account-creation path that skips it, and run the repro to confirm the mechanism regardless.

13. user (2026-07-06T07:56:01.669Z)

```
destinations: [(111, '@feed')]
routing_rules accounts: [(111,)]
delivery_attempts (account,status): [(111, 'delivered')]
content_items: [(111, '42'), (333, '99')]
```

14. assistant (2026-07-06T07:56:06.055Z)

The mechanism is confirmed at the SQL level: account 333 (channel + forwarded video 99, no filter_config) gets its content_item migrated (333/99 present) but NO destination, NO routing_rule, and NO delivery_attempt. Its forwarded history is silently dropped.

The critical question is whether such an account can actually exist in production. The finding claims "defaults-only user." Let me verify the legacy bootstrap always wrote a filter_config row ? this determines LIVE vs. purely-theoretical.

15. user (2026-07-06T07:56:07.816Z)

```
src\tg_video_filter\telethon_client.py:9:      -> db.insert_video(...) +
delivery.forward_to_target(..., target=cfg.delivery_target)
src\tg_video_filter\telethon_client.py:153:    delivery_target: str = "me",
src\tg_video_filter\telethon_client.py:164:    5. delivers a match to ``delivery_target`` (see
ADR 0005).
src\tg_video_filter\telethon_client.py:306:                target=delivery_target,
src\tg_video_filter\db.py:672:    ``filter_config_deprecated.delivery_target`` value, then
creates a default
src\tg_video_filter\db.py:689:    # Parse the JSON delivery_target, defaulting to "me".
src\tg_video_filter\db.py:694:        COALESCE(json_extract(config_json,
'$.delivery_target'), 'me'),
src\tg_video_filter\db.py:696:        WHEN COALESCE(json_extract(config_json,
'$.delivery_target'), 'me') = 'me'
src\tg_video_filter\db.py:718:        json_extract(fcd.config_json,
'$.delivery_target'), 'me'
src\tg_video_filter\db.py:1619:    delivery still targets the single ``delivery_target`` and
retried delivery
src\tg_video_filter\models.py:214:# destinatinons (backfilled from
filter_config.delivery_target), writes
src\tg_video_filter\models.py:223:    Generalizes the legacy ``FilterConfig.delivery_target``
field so one account
src\tg_video_filter\models.py:225:    per-account ``delivery_target``; the default destination
is the target used
src\tg_video_filter\models.py:347:    delivery_target: str = Field(
src\tg_video_filter\models.py:353:        @field_validator("delivery_target", mode="before")
src\tg_video_filter\models.py:355:        def _coerce_delivery_target(cls, v: object) -> str:
src\tg_video_filter\models.py:429:DEFAULT_FILTER_CONFIG = FilterConfig()
src\tg_video_filter\commands.py:143:    f"? **Delivery target:** `{cfg.delivery_target}`",
src\tg_video_filter\manager.py:154:        delivery_target=cfg.delivery_target,
src\tg_video_filter\manager.py:165:        delivery_target=cfg.delivery_target,
```

[Showing results with pagination = limit: 20]

16. assistant (2026-07-06T07:56:14.302Z)

Now let me assess reachability. Bootstrap (bootstrap.py:75) writes `DEFAULT\_FILTER\_CONFIG` for every onboarded user. So in the normal onboarding flow, every account has a filter_config row (with `delivery\_target` defaulting to "me"). This means the finding's "defaults-only user" scenario is actually COVERED ? a defaults-only user still has a filter_config row (containing `{delivery\_target: "me", ...}`), and migration 004 step 1 would create a "me"/saved destination for them.

Let me verify: does a defaults-only user have a filter_config row? bootstrap.py:75 always calls save_filter_config. Let me confirm save_filter_config writes a row unconditionally and check the legacy version. Actually the legacy DB at upgrade time was written by the OLD code. Let me check git or the schema to confirm the old bootstrap wrote it. The finding hinges on whether a "no filter_config row" account is realistic. Let me test the defaults-only case explicitly.

17. user (2026-07-06T07:56:30.494Z)

```
DEFAULT_FILTER_CONFIG json: {"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048,"exclude_gif":true,"exclude_round":true,"exclude_nosound":false,"delivery_mode":"forward","delivery_target":"me"}
destinations: [(555, 'me', 'saved')]
delivery_attempts: [(555, 'delivered')]
```

18. assistant (2026-07-06T07:56:38.840Z)

This is decisive. A **defaults-only user** (the finding's own example) HAS a filter_config row ? bootstrap.py:75 writes `DEFAULT\_FILTER\_CONFIG` (containing `delivery\_target: "me"`) for every onboarded account. Migration 004 correctly backfills them: destination `('me','saved')` and a delivery_attempt are created. So the finding's cited trigger ("defaults-only user") is REFUTED ? those users are handled.

The finding's real claim narrows to: an account with a filter_config **row entirely absent**. For that to happen, an account must exist in `users`/`channels`/`videos` but have NO `filter\_config` row. Given bootstrap always writes one atomically in the same function, this requires an anomalous DB state (manual deletion, a partial/failed onboarding, or a pre-bootstrap-era account). Let me confirm there's no code path that creates an account without filter_config, and check git history for whether legacy bootstrap always wrote it.

19. user (2026-07-06T07:56:40.936Z)

```
100abb2 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C)
323ee5a feat(domain): provider-neutral schema ? sources/content_items/policies (ADR 0011, Track B) (#35)
f6c9322 docs(adr): accept 0011 provider-neutral domain schema (Track B decision) (#34)
5ffc25b feat(identity): split users into accounts + telegram_sessions (ADR 0010, Track A) (#33)
0e21d36 revert: remove accidental file (tooling mistake ? will use issue-comment tool)
---grep upsert_account callers---
src/tg_video_filter/bootstrap.py:65:         await db.upsert_account(conn, Account(id=user.id,
telegram_id=user.id))
src/tg_video_filter/db.py:926:async def upsert_account(conn: aiosqlite.Connection, account:
Account) -> Account:
src/tg_video_filter/db.py:1057:     await upsert_account(conn, account)
src/tg_video_filter/db.py:1738:     "upsert_account",
Binary file src/tg_video_filter/__pycache__/bootstrap.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/db.cpython-313.pyc matches
```

20. assistant (2026-07-06T07:56:44.907Z)

The only account-creation path is `bootstrap.persist\_user` (bootstrap.py:63-78), which in the same transaction/function always calls `save\_filter\_config` at line 75. There is no path that creates an account+channels+videos without also writing a filter_config row.

Let me confirm db.py:1057 (the other upsert_account caller) ? to be thorough about whether it's an alternate onboarding path.

21. user (2026-07-06T07:56:45.863Z)

```
1040         )
1041         for r in rows
1042     ]
1043
1044
1045     # --- legacy User shims (backwards-compatible view over accounts + sessions) --
```

```

1046
1047
1048 async def upsert_user(conn: aioredis.Connection, user: User) -> User:
1049     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
1050
1051     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
1052     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
1053     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
1054     the manager can run them. ``User.enabled`` maps to ``session.enabled``.
1055     """
1056     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
1057     await upsert_account(conn, account)
1058     await upsert_telegram_session(
1059         conn,
1060         TelegramSession(
1061             account_id=user.id,
1062             phone=user.phone,
1063             session_path=f"user_{user.id}.session",
1064             enabled=user.enabled,

```

22. assistant (2026-07-06T07:57:02.938Z)

`upsert\_user` is a deprecated shim that creates account+session but NOT filter_config. However, this is a v1?ADR-0010 compatibility shim; it doesn't represent a live onboarding path that produces channels+videos. And critically, this creates an account with no channels/videos either, so it wouldn't have forwarded history to lose.

Let me now settle the verdict.

****Assessment:****

The finding's ****mechanism is real and confirmed**** (repro 1): migration 004 backfills destinations only from `filter\_config\_deprecated` (db.py:690-704). An account whose `filter\_config` row is entirely absent gets no destination ? no routing_rule ? no delivery_attempt, so its forwarded history in `videos\_deprecated.forwarded=1` is not backfilled (content_item survives, but delivery record is dropped). The test at test_db.py:1591 indeed only covers accounts with a filter_config row.

****However, the cited trigger is REFUTED:****

1. The finding names "defaults-only user (e.g. defaults-only user)" as the realistic production trigger. Repro 2 proves this is wrong ? a defaults-only user HAS a filter_config row (bootstrap.py:75 writes `DEFAULT\_FILTER\_CONFIG` with `delivery\_target: "me"` for every onboarded account), and migration 004 correctly backfills it to a `('me', 'saved')` destination plus a delivery_attempt. Defaults-only users are handled.
2. For the actual defect to trigger, an account must have channels+forwarded videos but a ****completely missing**** filter_config row. The only account-creation path that produces channels/videos is `bootstrap.persist\_user`, which writes the filter_config row in the same function (bootstrap.py:63-78). There is no code path that produces the required anomalous state. It could only arise from manual DB tampering or a partial/crashed onboarding ? not a normal production upgrade.

So the described failure scenario ("any account... e.g. a defaults-only user... loses all its forwarded history") does not occur in practice. The severity claim of "medium | live" is overstated: the mechanism is latent behind an unreachable-in-normal-operation precondition. It's a real untested edge (test coverage gap + no defensive default for a NULL delivery_target vs. missing row), but not a data-drop that ships for any normal account.

Given the finding's stated failure scenario is refuted by the realistic trigger it names, and the residual concern is a low-severity untested edge case requiring an anomalous DB, I'll mark this REFUTED with a note, corrected severity low.

23. user (2026-07-06T07:57:15.491Z)

Structured output provided successfully